

02 — Product Requirements Document (PRD)

Interpretable Machine Learning for Discovering Hidden Regulatory Switches in Non-Coding DNA

Field	Value
Document ID	PRD-001
Version	1.0
Status	Active
Created	2026-06-07
Related docs	Charter , Technical Design , Research Design

1. Overview

This document specifies the product requirements for an interpretable ML research pipeline that predicts regulatory activity in non-coding DNA and explains which sequence features contribute to that prediction. The "product" is a research workbench — initially a collection of Kaggle notebooks and supporting modules — that evolves into a documented, modular platform for regulatory genomics research.

The system consumes publicly annotated genomic regions (e.g., from ENCODE), extracts DNA sequences, engineers or learns features, trains interpretable classifiers, and produces both predictions and interpretability outputs (motif importance rankings, feature contributions, attribution maps).

2. User Types / Researcher Personas

Persona	Description	Primary needs
Core Builder	CS undergraduate on the project team. Strong in Python/ML, learning genomics.	Clear task definitions, runnable notebooks, modular code, fast iteration.
Research Advisor	Faculty member or senior researcher providing scientific oversight.	Sound methodology, interpretable results, publication-quality figures, reproducibility.
Future Contributor	Student joining the project mid-stream.	Onboarding docs, protected scope, clear code structure, safe contribution paths.
External Researcher	Computational biologist or bioinformatician evaluating or reusing the pipeline.	Reproducible results, well-documented data processing, downloadable artifacts, honest claims.
Demo Viewer	Non-specialist viewing a project presentation or poster.	Clear visualizations, intuitive explanations of regulatory switch concept, no jargon overload.

3. Problem to Solve

Researchers need tools that can:

1. **Scan** large catalogs of non-coding genomic regions and predict which ones have regulatory activity.
2. **Explain** predictions by attributing importance to specific sequence features (motifs, k-mers, local patterns).
3. **Compare** interpretability outputs across cell types, model families, and experimental conditions.

4. **Operate** within student-accessible compute constraints (Kaggle notebooks, no multi-GPU clusters).

Existing large-scale genomic models provide strong accuracy but lack interpretability and require infrastructure beyond the team's access. Existing classical approaches are interpretable but may miss complex patterns. This project occupies the middle ground.

4. Use Cases

UC-1: Regulatory Region Classification

Actor: Core Builder / Research Advisor

Flow: User provides a set of non-coding genomic coordinates → system extracts sequences → system runs trained classifier → system returns regulatory/non-regulatory predictions with confidence scores.

Output: Prediction table (region ID, prediction label, probability, cell-type context).

UC-2: Motif Importance Analysis

Actor: Core Builder / Research Advisor

Flow: User selects a predicted-regulatory region → system runs interpretability module → system identifies top contributing sequence features → system cross-references with known motif databases.

Output: Ranked feature importance list, motif logos (if applicable), overlap with JASPAR/HOCOMOCO motifs.

UC-3: Cross-Cell-Type Comparison

Actor: Research Advisor / External Researcher

Flow: User selects two or more cell types → system runs classifiers trained per cell type → system compares which motifs are important in each → system highlights cell-type-specific regulatory patterns.

Output: Comparative motif importance table, differential feature heatmap.

UC-4: Model Benchmarking

Actor: Core Builder

Flow: User defines a set of models (logistic regression, random forest, CNN, etc.) → system trains all on the same data split → system produces a standardized evaluation report.

Output: AUROC, AUPRC, accuracy, F1 per model; interpretability comparison matrix.

UC-5: Reproducible Experiment Execution

Actor: Core Builder / Future Contributor

Flow: User opens a Kaggle notebook → notebook loads data from versioned dataset → notebook trains model with logged hyperparameters → notebook saves metrics and artifacts to designated output.

Output: Logged experiment, saved model, reproducible notebook.

UC-6: Research Report Generation

Actor: Research Advisor

Flow: User triggers report generation → system aggregates experiment results, top motifs, model comparisons → system produces a structured markdown or HTML report.

Output: Formatted research summary with tables, figures, and interpretability highlights.

5. Core Features

5.1 Data Pipeline

Feature	Description	Priority
F001-01	Download reference GENCODE transcripts and associated genomic features	P1 (Global Basis)
F001-02	Extract DNA sequences for annotated regions from reference genome	P1
F001-03	Download regulatory regions (e.g., ChIP-seq peaks) from public databases	P1
F001-04	Extract genomic coordinates	P1
F001-05	Compute cross-frequency features for stability	P1
F001-06	Compute GC content, sequence complexity, and associated features	P1 (Global Basis)
F001-07	Download known regulatory motifs from databases	P1
F001-08	Cache preprocessed data to avoid redundant computation	P1
F001-09	Support multiple genome assemblies (hg38, pan-genome, etc.)	P2 (Global Basis)

5.2 Modeling

Feature	Description	Priority
F-ML-01	Logistic regression on k-mer/motif features	P0
F-ML-02	Random forest on k-mer/motif features	P0
F-ML-03	Gradient-boosted trees (XGBoost/LightGBM) on engineered features	P0
F-ML-04	Shallow 1D CNN on one-hot encoded sequences	P1
F-ML-05	Lightweight attention-based model (only if compute allows)	P2
F-ML-06	Pretrained embedding + simple classifier (Stage 3 exploration)	P2
F-ML-07	Hyperparameter tuning with logged search	P1
F-ML-08	Ensemble or stacking of multiple model families	P2

5.3 Interpretability

Feature	Description	Priority
F-IX-01	Feature importance from tree-based models (Gini / permutation importance)	P0
F-IX-02	Logistic regression coefficient analysis	P0
F-IX-03	k-mer importance ranking and visualization	P0
F-IX-04	SHAP values for tree-based and linear models	P1
F-IX-05	Saliency maps / integrated gradients for CNN models	P1
F-IX-06	Motif enrichment analysis on high-importance regions	P1
F-IX-07	Attention visualization (only if attention model is used)	P2
F-IX-08	Cross-cell-type motif comparison	P1
F-IX-09	Automated motif-to-TF mapping via JASPAR/HOCOMOCO lookup	P2

5.4 Evaluation & Reporting

Feature	Description	Priority
F-EV-01	AUROC, AUPRC, accuracy, precision, recall, F1 computation	P0
F-EV-02	Confusion matrix and classification report generation	P0
F-EV-03	Cross-validation support (stratified k-fold)	P0
F-EV-04	Chromosome-holdout evaluation (to prevent spatial leakage)	P1
F-EV-05	Model comparison dashboard (table or notebook-rendered)	P1
F-EV-06	Automated research report generation (markdown)	P2

5.5 Experiment Management

Feature	Description	Priority
F-EM-01	Experiment naming convention enforcement	P0
F-EM-02	Hyperparameter and metric logging (CSV or JSON)	P0
F-EM-03	Model artifact saving with version tags	P1
F-EM-04	Notebook-to-script conversion support	P1
F-EM-05	Integration with Weights & Biases or MLflow (optional)	P2

6. Data Inputs

Input	Format	Source	Size estimate
Regulatory region annotations	BED / narrowPeak	ENCODE portal	~50–500 MB per experiment type
Reference genome (hg38)	FASTA	UCSC / NCBI	~3.1 GB (can use per-chromosome files)
Known TF motifs	PFM / MEME	JASPAR / HOCOMOCO	~10 MB
Gene annotations	GTF / GFF3	GENCODE	~50 MB
(Later) Expression data	TSV / HDF5	GTEX / GEO	Variable

7. Outputs

Output	Format	Consumer
Prediction table	CSV / Parquet	All personas
Feature importance rankings	CSV + plots (PNG/SVG)	Builder, Advisor
Motif analysis report	Markdown + figures	Advisor, External Researcher
Model performance summary	CSV + plots	Builder, Advisor
Trained model artifacts	Pickle / ONNX / PyTorch checkpoint	Builder
Experiment logs	JSON / CSV	Builder
Interpretability visualizations	PNG / SVG / HTML	All personas
Research report	Markdown / PDF	Advisor, External Researcher

8. Functional Requirements

ID	Requirement	Comments
FR-01	The system shall accept FASTQ files as input and output BAM files.	F-0001, F-0002
FR-02	The system shall generate a report of the model's performance metrics.	F-0003
FR-03	The system shall generate a report of the model's feature importance metrics.	F-0004
FR-04	The system shall generate a report of the model's motif analysis results.	F-0005, F-0006
FR-05	The system shall generate a report of the model's performance metrics across different experimental conditions.	F-0007, F-0008
FR-06	The system shall generate a report of the model's performance metrics across different experimental conditions.	F-0009
FR-07	The system shall generate a report of the model's performance metrics across different experimental conditions.	F-0010
FR-08	The system shall generate a report of the model's performance metrics across different experimental conditions.	F-0011
FR-09	The system shall generate a report of the model's performance metrics across different experimental conditions.	F-0012
FR-10	The system shall generate a report of the model's performance metrics across different experimental conditions.	F-0013

9. Non-Functional Requirements

ID	Requirement	Metric
NFR-01	Kaggle Compatibility: All P0 and P1 features must execute within a single Kaggle notebook session.	Runs within 16 GB RAM, 1 GPU, 12 hr session
NFR-02	Reproducibility: Given the same data version and random seed, results must be reproducible.	Bit-identical metrics on re-run
NFR-03	Modularity: Code must be organized into reusable modules, not monolithic notebooks.	≥ 5 importable modules
NFR-04	Documentation: Every public function must have a docstring; every module must have a README.	100% public API documentation
NFR-05	Performance: Full pipeline (data load → train → evaluate → interpret) must complete in < 4 hours on Kaggle GPU.	Wall-clock time
NFR-06	Data integrity: No train/test leakage by region overlap or chromosome proximity.	Verified by chromosome-holdout scheme
NFR-07	Extensibility: Adding a new model family must require ≤ 1 new file and ≤ 20 lines of integration code.	Measured by adding a dummy model

10. Assumptions

ID	Assumption
A-01	ENCODE data remains publicly available under its current data use policy.
A-02	Kaggle free-tier GPU access remains available with at least current quotas.
A-03	The hg38 reference genome is the appropriate assembly for initial work.
A-04	k-mer frequencies ($k \leq 6$) capture sufficient signal for a baseline regulatory classifier.
A-05	Known TF motif databases (JASPAR, HOCOMOCO) are sufficient for motif cross-referencing.
A-06	Chromosome-holdout is an acceptable strategy for reducing spatial leakage.
A-07	The team has sufficient Python/ML skill to implement the pipeline without extensive bioinformatics mentorship.

11. Dependencies

Dependency	Type	Risk if unavailable
ENCODE portal API / downloads	Data	Cannot build dataset; use cached snapshots
hg38 FASTA (UCSC/NCBI)	Data	Cannot extract sequences; pre-extract and cache
Kaggle GPU runtime	Compute	Fall back to CPU-only or Google Colab
Python scientific stack (NumPy, pandas, scikit-learn, XGBoost)	Software	Core dependency; stable and well-maintained
Biopython / pysam / pybedtools	Software	Needed for genomic data handling; installable on Kaggle
SHAP library	Software	Needed for interpretability; installable on Kaggle
PyTorch (for CNN models)	Software	Available on Kaggle by default
JASPAR motif database	Data	Needed for motif cross-referencing; downloadable

12. Acceptance Criteria

Phase 1 (Baseline)

Criterion	Verification
AC-1.1	A regulatory vs. non-regulatory classifier achieves AUROC ≥ 0.75 on ENCODE-derived test data.
AC-1.2	Feature importance output identifies at least 5 biologically plausible k-mers or motifs.
AC-1.3	The pipeline runs end-to-end in a single Kaggle notebook within the session time limit.
AC-1.4	All experiments are logged with hyperparameters, metrics, and data version.
AC-1.5	A negative-region generation strategy is implemented and documented.

Phase 2 (Research-Grade)

Criterion	Verification
AC-2.1	At least 3 model families benchmarked on the same data split with standardized metrics.
AC-2.2	AUROC ≥ 0.80 achieved by at least one model.
AC-2.3	SHAP or saliency-based interpretability available for at least one model.
AC-2.4	Cross-cell-type experiment completed for at least 2 cell types.
AC-2.5	Chromosome-holdout evaluation implemented and reported.

Phase 3 (Extension)

Criterion	Verification
AC-3.1	Pretrained genomic embeddings (e.g., from DNABERT-2 or Nucleotide Transformer) integrated as features.
AC-3.2	Accuracy comparison between pretrained-embedding classifiers and interpretable baselines documented.
AC-3.3	At least one interpretability mechanism applied to the pretrained pipeline.

End of Document 02 — Product Requirements Document

Next: [03 — Technical Design Document](#)